

OmniCart

A Relational E-Commerce Database Management System

with AI-Powered Product Recommendations

DATABASE SYSTEMS PROJECT REPORT

Course	CS 2005: Database Systems
Instructor	Sir Shoaib Rauf
Project Title	OmniCart: Relational E-Commerce DBMS
Group Leader	Rayyan Aamir — 24K-0687
Group Member	Ahmed Shah Rashdi — 24K-0709
Group Member	Muhammad Usaid Khan — 24K-0832
Semester	Spring 2026
Submission Date	11th March, 2026

1. Introduction and System Overview

OmniCart is a full-stack, web-based E-Commerce Database Management System designed to manage the complete lifecycle of an online retail platform. The system provides a structured relational database backend built on MySQL that handles all core e-commerce operations including product cataloging, order processing, customer management, inventory tracking, and seller administration.

What distinguishes OmniCart from a conventional e-commerce system is the seamless integration of Artificial Intelligence into the database-driven platform. The system leverages Machine Learning

algorithms trained on the Amazon Product Reviews Dataset to deliver personalized product recommendations, while an n8n automation layer handles all triggered communications such as order confirmations, cart abandonment reminders, and weekly personalized deal digests.

The system is designed around three distinct user roles — Customer, Seller, and Administrator — each interacting with the database through a dedicated web-based portal. All database interactions are managed through a Python Flask backend with SQLAlchemy ORM, ensuring clean separation between business logic, the AI layer, and the data layer.

1.1 Key Objectives

- Design and implement a normalized 11-table relational MySQL database covering all e-commerce entities.
- Build 3 role-based stored procedures encapsulating critical business transactions.
- Implement 5 complex SQL queries with JOINS, aggregates, subqueries, and built-in functions.
- Integrate AI/ML recommendation and sentiment analysis pipelines backed by the database.
- Automate event-driven communications using 4 n8n workflows triggered via Flask webhooks.

2. System Functionality

2.1 Customer Functions

- User Registration and Authentication: JWT-based registration and login with bcrypt password hashing.
- Product Browsing and Search: Browse by category, search by keyword, filter by price, brand, and rating.
- AI-Powered Recommendations: Personalized product recommendations based on purchase history using collaborative filtering.
- Shopping Cart Management: Add, update, and remove items; cart data is persisted in the database.
- Order Placement with Transaction: Atomic SQL transaction — inserts the order, deducts stock, and clears the cart simultaneously.
- Review Submission: Star ratings and written reviews; sentiment analysis runs automatically and stores scores.
- Order History: Full order history with product details, status, and invoice summaries.

2.2 Seller Functions

- Product Management: Add, update, and manage listings including prices and stock levels.
- AI Description Generator: Request GPT-4 generated product descriptions when adding new products.
- Sales Analytics: Revenue reports, best-selling products, and low-stock alerts.
- Order Fulfillment Tracking: View and update status of orders containing their products.

2.3 Administrator Functions

- Full CRUD Operations: Complete Create, Read, Update, and Delete access over all tables.
- User Management: Activate, deactivate, or delete customer and seller accounts via stored procedure.
- Platform Analytics Dashboard: Top-selling products, most active customers, sentiment scores by category, revenue trends.
- Category Management: Manage the product category hierarchy in the Categories table.
- Recommendation Audit: View the Recommendations table to audit which products were recommended with what confidence score.

2.4 Automated System Functions

- Order Confirmation: n8n triggers an automated HTML confirmation email immediately after every order.
- Cart Abandonment Reminder: n8n checks the Cart table every 24 hours and emails customers with abandoned items.
- Weekly Deals Digest: Every Monday, n8n queries the Recommendations table and sends personalized HTML emails with top-5 picks.
- Low Stock Alert: n8n monitors Products and alerts sellers when stock drops below 10 units.

3. Database Schema

OmniCart uses a 11-table relational schema (omnicart_db) built on MySQL 8.0. Every table uses surrogate integer primary keys with AUTO_INCREMENT. Foreign keys enforce referential integrity with appropriate ON DELETE and ON UPDATE rules. Indexes are applied on all foreign key columns to optimize JOIN performance.

3.1 Table Summary

Table	Primary Key	Key Attributes
-------	-------------	----------------

customers	customer_id	name, email, password_hash, phone, address, date_of_birth, is_active, created_at
vendors	vendor_id	name, email, password_hash, store_name, phone, is_approved, is_active, created_at
categories	category_id	name, description, parent_category_id (self-referencing FK)
products	product_id	vendor_id, category_id, name, description, price, stock_qty, brand, is_active
orders	order_id	customer_id, status (ENUM), total_amount, placed_at
order_items	item_id	order_id, product_id, quantity, unit_price
payments	payment_id	order_id (UNIQUE), method (ENUM), status (ENUM), amount, paid_at
reviews	review_id	customer_id, product_id, rating (1–5), comment, sentiment_score (0.00–1.00)
recommendations	rec_id	customer_id, product_id, score (0.0000–1.0000), explanation, generated_at
cart	cart_id	customer_id, product_id, quantity, added_at
admins	admin_id	name, email, password_hash, created_at

3.2 Key Constraints & Integrity Rules

- email columns have CHECK (email LIKE '%@%.%') across customers, vendors, and admins.
- phone columns enforce Pakistani format: CHECK (phone REGEXP '^03[0-9]{2}-[0-9]{7}\$').
- orders.status is ENUM('pending','confirmed','shipped','delivered','cancelled').
- payments.method is ENUM('credit_card','debit_card','cash_on_delivery','bank_transfer').
- reviews.rating has CHECK (rating BETWEEN 1 AND 5); reviews.sentiment_score CHECK (0.00–1.00).
- recommendations.score has CHECK (score BETWEEN 0.0000 AND 1.0000).
- cart and reviews use UNIQUE KEY on (customer_id, product_id) preventing duplicate entries.
- payments.order_id is UNIQUE — one payment record per order.

3.3 Foreign Key Relationships

Child Table → Column	References	ON DELETE	ON UPDATE
products → vendor_id	vendors(vendor_id)	CASCADE	CASCADE
products → category_id	categories(category_id)	RESTRICT	CASCADE
orders → customer_id	customers(customer_id)	CASCADE	CASCADE

order_items → order_id	orders(order_id)	CASCADE	CASCADE
order_items → product_id	products(product_id)	RESTRICT	CASCADE
payments → order_id	orders(order_id)	CASCADE	CASCADE
reviews → customer_id	customers(customer_id)	CASCADE	CASCADE
reviews → product_id	products(product_id)	CASCADE	CASCADE
recommendations → customer_id	customers(customer_id)	CASCADE	CASCADE
recommendations → product_id	products(product_id)	CASCADE	CASCADE
cart → customer_id	customers(customer_id)	CASCADE	CASCADE
cart → product_id	products(product_id)	CASCADE	CASCADE
categories → parent_category_id	categories(category_id)	SET NULL	CASCADE

4. Database Views

Three SQL Views are defined to simplify complex reporting queries and provide pre-joined data to the application layer.

4.1 CustomerOrderHistory

Joins customers → orders → order_items → products. Returns full order history for any customer including product names, quantities, unit prices, subtotals, order status, and placement timestamp.

```
CREATE VIEW CustomerOrderHistory AS
SELECT c.name AS customer_name, c.email, o.order_id,
       p.name AS product_name, oi.quantity, oi.unit_price,
       (oi.quantity * oi.unit_price) AS subtotal,
       o.status, o.placed_at
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

4.2 TopRatedProducts

Aggregates reviews by product with HAVING avg_rating >= 3.5. Joins products → reviews → categories. Returns product name, brand, category, average rating, total review count, price, and stock.

```
CREATE VIEW TopRatedProducts AS
```

```

SELECT p.product_id, p.name AS product_name, p.brand,
       c.name AS category, ROUND(AVG(r.rating), 1) AS avg_rating,
       COUNT(r.review_id) AS total_reviews, p.price, p.stock_qty
FROM products p
JOIN reviews r ON p.product_id = r.product_id
JOIN categories c ON p.category_id = c.category_id
GROUP BY p.product_id
HAVING avg_rating >= 3.5
ORDER BY avg_rating DESC;

```

4.3 VendorSalesSummary

Aggregates vendor-level sales data. LEFT JOINs vendors → products → order_items → reviews. Returns total units sold, total revenue, and average product rating per vendor-product combination. COALESCE handles vendors with zero sales.

```

CREATE VIEW VendorSalesSummary AS
SELECT v.vendor_id, v.store_name, p.name AS product_name,
       COALESCE(SUM(oi.quantity), 0) AS total_units_sold,
       COALESCE(SUM(oi.quantity * oi.unit_price), 0) AS total_revenue,
       COALESCE(ROUND(AVG(r.rating), 1), 0.0) AS avg_product_rating
FROM vendors v
JOIN products p ON v.vendor_id = p.vendor_id
LEFT JOIN order_items oi ON p.product_id = oi.product_id
LEFT JOIN reviews r ON p.product_id = r.product_id
GROUP BY v.vendor_id, p.product_id;

```

5. Stored Procedures

Three stored procedures encapsulate the most critical business transactions in OmniCart. Each maps directly to a user role and is invoked from the Flask backend via SQLAlchemy using CALL statements.

5.1 PlaceOrder — Customer Role

Procedure	PlaceOrder(IN p_customer_id INT, IN p_cart_items JSON)
Role	Customer
Purpose	Atomic order placement — inserts order, deducts stock for each cart item, clears cart
Transaction	Wrapped in START TRANSACTION / COMMIT with ROLLBACK on any SQLEXCEPTION
Called via	POST /api/orders → db.session.execute(text('CALL PlaceOrder(:cid, :items)'))

Business logic inside the procedure:

- Validates customer exists and is active; signals error if not.
- Checks cart is non-empty via `JSON_LENGTH(p_cart_items)`.
- Iterates each cart item using a `WHILE` loop: locks the product row with `FOR UPDATE`, checks `is_active`, checks `stock_qty >= quantity`.
- Inserts `order_items` record and deducts stock with `UPDATE products SET stock_qty = stock_qty - v_quantity`.
- Updates `orders.total_amount` to the computed sum, then deletes the customer's cart rows.

5.2 UpdateProductInventory — Seller Role

Procedure	UpdateProductInventory(IN p_product_id INT, IN p_vendor_id INT, IN p_new_stock INT, IN p_new_price DECIMAL)
Role	Seller / Vendor
Purpose	Securely update stock quantity and price for a product owned by the calling vendor
Auth Check	Validates product belongs to the calling vendor; raises <code>SQLSTATE 45000</code> if ownership mismatch
Validation	Signals error if <code>p_new_stock < 0</code> OR <code>p_new_price < 0</code>
Called via	<code>PUT /api/seller/products/<id> → CALL UpdateProductInventory(...)</code>

5.3 DeactivateUser — Admin Role

Procedure	DeactivateUser(IN p_admin_id INT, IN p_user_id INT, IN p_user_type VARCHAR(20), IN p_reason VARCHAR(255))
Role	Administrator
Purpose	System-wide deactivation of a customer or vendor account with cascading side-effects
Transaction	Wrapped in <code>START TRANSACTION / COMMIT</code>
Customer	Sets <code>is_active = FALSE</code> ; cancels all pending orders for that customer
Vendor	Sets <code>is_active = FALSE</code> ; marks all vendor products as <code>is_active = FALSE</code>
Called via	<code>POST /api/admin/deactivate → CALL DeactivateUser(...)</code>

5.4 ProcessPayment — Customer Role

Atomically inserts a payment record, validates the amount matches order total, prevents double-payment, and updates order status from pending to confirmed. Supports methods: `credit_card`, `debit_card`, `cash_on_delivery`, `bank_transfer`. Full race-condition protection via `FOR UPDATE` lock on the order row.

5.5 CancelOrder — Customer Role

Reverses PlaceOrder: restores stock quantities for all order items, cancels the order, and handles payment reversal (refunded if payment was completed, failed if payment was pending). Only pending or confirmed orders can be cancelled.

6. Complex SQL Queries

Five complex queries are implemented in queries.sql and exposed as named functions through the Flask backend. Each demonstrates multiple advanced SQL concepts.

Query 1 — Customer Full Order History (Customer Role)

4-table JOIN: customers → orders → order_items → products. Filtered by customer_id. Returns line-item detail with computed subtotal column.

```
SELECT o.order_id, o.status, o.placed_at,
       p.name AS product_name, p.brand,
       oi.quantity, oi.unit_price,
       (oi.quantity * oi.unit_price) AS subtotal,
       o.total_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE c.customer_id = :customer_id
ORDER BY o.placed_at DESC;
```

Query 2 — Platform Revenue Report by Month (Admin Role)

Uses DATE_FORMAT built-in function to group by year-month. Aggregates with COUNT, SUM, AVG, and ROUND. Filters out cancelled orders. Returns last 12 months of revenue data.

```
SELECT DATE_FORMAT(o.placed_at, '%Y-%m') AS month,
       COUNT(o.order_id) AS total_orders,
       ROUND(SUM(o.total_amount), 2) AS total_revenue,
       ROUND(AVG(o.total_amount), 2) AS avg_order_value
FROM orders o
WHERE o.status != 'cancelled'
GROUP BY DATE_FORMAT(o.placed_at, '%Y-%m')
ORDER BY DATE_FORMAT(o.placed_at, '%Y-%m') DESC
LIMIT 12;
```

Query 3 — Top Customers by Spend (Admin Role)

Multi-table JOIN with GROUP BY, HAVING filter on order count, and a correlated subquery to count reviews per customer. Returns top 20 customers by total spend.

```
SELECT c.customer_id, c.name, c.email,
       COUNT(DISTINCT o.order_id) AS total_orders,
       ROUND(SUM(o.total_amount), 2) AS total_spent,
       ROUND(AVG(o.total_amount), 2) AS avg_order_value,
       (SELECT COUNT(*) FROM reviews r
        WHERE r.customer_id = c.customer_id) AS review_count
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.status != 'cancelled'
GROUP BY c.customer_id, c.name, c.email
HAVING COUNT(DISTINCT o.order_id) >= 1
ORDER BY total_spent DESC LIMIT 20;
```

Query 4 — Vendor Sales Performance (Admin / Seller Role)

LEFT JOIN chain: vendors → products → order_items → reviews. COALESCE handles vendors with zero sales. Aggregates units sold, total revenue, average rating, and total reviews per vendor.

```
SELECT v.vendor_id, v.store_name, v.name AS vendor_name,
       COUNT(DISTINCT p.product_id) AS product_count,
       COALESCE(SUM(oi.quantity), 0) AS total_units_sold,
       COALESCE(ROUND(SUM(oi.quantity * oi.unit_price), 2), 0) AS total_revenue,
       COALESCE(ROUND(AVG(r.rating), 1), 0.0) AS avg_rating
FROM vendors v
LEFT JOIN products p ON v.vendor_id = p.vendor_id
LEFT JOIN order_items oi ON p.product_id = oi.product_id
LEFT JOIN reviews r ON p.product_id = r.product_id
WHERE v.is_active = TRUE
GROUP BY v.vendor_id, v.store_name, v.name, v.is_approved
ORDER BY total_revenue DESC;
```

Query 5 — Category Sentiment & Rating Analysis (Admin Role)

Joins categories → products → reviews. Computes average rating and average sentiment_score per category. Correlated subquery counts distinct products with positive sentiment (score >= 0.7). Links the AI sentiment pipeline directly to DB reporting.

```
SELECT c.category_id, c.name AS category_name,
       COUNT(DISTINCT p.product_id) AS product_count,
       COUNT(DISTINCT r.review_id) AS total_reviews,
       COALESCE(ROUND(AVG(r.rating), 2), 0.0) AS avg_rating,
       COALESCE(ROUND(AVG(r.sentiment_score), 2), 0.0) AS avg_sentiment,
       (SELECT COUNT(DISTINCT p2.product_id)
        FROM products p2 JOIN reviews r2 ON p2.product_id = r2.product_id
        WHERE p2.category_id = c.category_id
        AND r2.sentiment_score >= 0.7) AS positive_products
FROM categories c
JOIN products p ON c.category_id = p.category_id
JOIN reviews r ON p.product_id = r.product_id
WHERE p.is_active = TRUE
GROUP BY c.category_id, c.name
ORDER BY avg_sentiment DESC;
```

7. AI & Machine Learning Layer

OmniCart integrates three AI components that are tightly coupled with the database schema. Each component reads from or writes to specific database tables, making the AI layer an extension of the data architecture.

7.1 SVD Collaborative Filtering (train_svd.py)

Trains a Singular Value Decomposition (SVD) model using the Surprise library on the Amazon Product Reviews dataset (UserId, ProductId, Score). The model learns latent user and product factors from implicit purchase behavior.

Algorithm	SVD (Matrix Factorization) — <code>n_factors=50, n_epochs=20</code>
Training Data	Amazon Reviews CSV, sampled to 200,000 rows after deduplication
Evaluation	3-fold cross-validation; RMSE and MAE reported
Output	<code>svd_model.pkl</code> + <code>product_map.pkl</code> saved to <code>ml/</code> directory
DB Integration	Reads orders + order_items to get customer purchase history; writes top recommendations to recommendations table

7.2 TF-IDF Content-Based Filtering (train_tfidf.py)

Builds a content-based recommendation engine using TF-IDF vectorization and cosine similarity on product text features. Products are loaded directly from the OmniCart database.

Algorithm	TfidfVectorizer (<code>max_features=5000, ngram_range=(1,2)</code>) + cosine_similarity
Features	product name (×2 weight), brand (×2 weight), category, parent category, description
DB Source	SELECT from products JOIN categories WHERE is_active = TRUE
Output	<code>tfidf_model.pkl</code> with vectorizer, cosine_sim matrix, and product index maps
Similarity	Cosine similarity matrix (<code>n_products × n_products</code>) for instant lookup

7.3 Sentiment Analysis Pipeline (train_sentiment.py)

Uses TextBlob to analyze review text and compute a sentiment polarity score normalized to 0.0–1.0. Scores are stored in `reviews.sentiment_score` and drive the Category Sentiment query (Query 5).

Library	TextBlob — polarity score <code>[-1.0, 1.0]</code> → normalized to <code>[0.0, 1.0]</code>
Labels	positive (≥ 0.6), neutral (<code>0.4–0.6</code>), negative (< 0.4)

Validation	Tested on 5,000 Amazon Reviews; accuracy compared against Score (1–5) ground truth
DB Write	UPDATE reviews SET sentiment_score = :score WHERE review_id = :id
Integration	Automatically called on every POST /api/reviews; scores stored immediately

7.4 Hybrid Recommendation Engine

Combines SVD collaborative filtering and TF-IDF content-based filtering using a weighted score: $\text{final_score} = 0.6 \times \text{CF_score} + 0.4 \times \text{CB_score}$. Products with $\text{sentiment_score} < 0.4$ are filtered out before ranking. Top-5 recommendations are inserted into the recommendations table with GPT-4 generated explanations.

8. n8n Automation Workflows

OmniCart implements 4 automation workflows using n8n. Each workflow is exported as a JSON file and imported directly into n8n. Workflows are triggered either by Flask webhooks or CRON schedules. All email delivery uses an SMTP credential configured in n8n.

Workflow	Trigger	n8n File	Description
Order Confirmation	Webhook POST /order-placed	order-confirmation.json	Sends formatted HTML order receipt to customer
Cart Abandonment Reminder	CRON every 24 hours	cart_abandonment_reminder.json	Loops over abandoned carts and emails each customer
Low Stock Alert	CRON every 6 hours	low_stock_alert.json	Emails vendor when any product stock < 10 units
Weekly Deals Digest	CRON every Monday 9 AM	weekly_deals_digest.json	Sends personalized AI recommendation emails to all customers

8.1 Common Workflow Pattern

Three of the four workflows share an identical pattern: Webhook → IF (count > 0) → Code (JavaScript map) → Loop Over Items → Email. The IF node prevents processing when there are no results. The Code node flattens the JSON array payload. The Loop node batches items one at a time through the Email node.

The Order Confirmation workflow is simpler — Webhook → Email — because it processes a single order payload immediately after placement with no batching required.

8.2 Flask Integration

Flask triggers workflows by calling `requests.post()` to the n8n webhook URL after relevant database operations. The order confirmation webhook fires immediately after a successful `CALL PlaceOrder(...)`. Cart abandonment and low-stock endpoints are polled by n8n CRON; Flask provides the data via dedicated GET endpoints.

9. Technologies Used

9.1 Database Technologies

Technology	Purpose
MySQL 8.0	Primary RDBMS — all 11 tables, stored procedures, views, transactions
MySQL Workbench	ERD design, query testing, schema management, stored procedure development
SQL	DDL (CREATE, ALTER), DML (INSERT, UPDATE, DELETE), Views, Transactions, Stored Procedures

9.2 Backend Technologies

Technology	Purpose
Python 3.11	Primary backend programming language
Flask	Lightweight web framework for REST API (20+ endpoints)
SQLAlchemy	ORM for database interaction and model definition
Flask-JWT-Extended	JWT-based authentication and role-based access control
Flask-Bcrypt	Password hashing and verification
PyMySQL	MySQL connector for SQLAlchemy

OpenAI API (GPT-4)	AI-generated product descriptions and recommendation explanations
Scikit-surprise	SVD collaborative filtering model training and evaluation
Scikit-learn	TF-IDF vectorization and cosine similarity computation
TextBlob / NLTK	Review sentiment analysis — polarity scoring
n8n	Workflow automation for emails, alerts, and scheduled tasks

9.3 Frontend Technologies

Technology	Purpose
React.js	Component-based SPA framework for three separate role-based portals
Tailwind CSS	Responsive layout and consistent UI styling across all pages
Chart.js	Analytics charts for admin (revenue, orders) and seller (sales) dashboards
Axios	HTTP client for API communication with JWT token interceptor

10. User Roles & Permissions

Role	Portal / Access Level	Key Database Operations
Customer	Customer Portal — JWT-authenticated; can only access own data	SELECT products/categories; INSERT/DELETE cart; CALL PlaceOrder; CALL CancelOrder; CALL ProcessPayment; INSERT reviews; SELECT own orders/recommendations
Seller / Vendor	Seller Portal — JWT with vendor role check; restricted to own products	INSERT/UPDATE own products via CALL UpdateProductInventory; SELECT own orders (via order_items); SELECT VendorSalesSummary view
Administrator	Admin Portal — full access; all tables visible	CALL DeactivateUser; full CRUD on all tables; execute all 5 complex queries; approve vendors; audit recommendations table

11. System Scope

11.1 In Scope

- Fully functional web-based e-commerce platform with browser-accessible portals.
- 11-table normalized MySQL database covering all core entities.
- Complete user registration, authentication, and role-based access for all three roles.
- Full product lifecycle management: listing, browsing, purchasing, and reviewing.
- AI recommendation engine using SVD + TF-IDF Hybrid scoring.
- Sentiment analysis on reviews with scores persisted in the database.
- GPT-4 explanations for recommendations and AI-generated product descriptions.
- Atomic transaction handling with rollback guarantees for order placement.
- Database Views for order history and top-rated products reporting.
- 4 n8n automation workflows with SMTP email delivery.
- Admin analytics dashboard powered by advanced SQL queries.

11.2 Out of Scope

- Real payment gateway integration (Stripe, PayPal) — payments are simulated.
- Mobile application development — web-browser based only.
- Real-time chat or customer support messaging.
- Shipping and logistics tracking with third-party courier APIs.
- Multi-language or multi-currency support.
- Social media OAuth login — standard email/password only.

12. Assumptions

- Each product belongs to exactly one vendor.
- A customer can place multiple orders; each order can contain multiple products via `order_items`.
- Sentiment scores range from 0.0 (most negative) to 1.0 (most positive) and are computed automatically on review submission.
- The AI recommendation engine is pre-trained on the Amazon Product Reviews Dataset before system deployment; the trained models are loaded at runtime.
- n8n is hosted locally during development and connects to the Flask backend via HTTP webhooks.

- Product prices are stored in PKR (Pakistani Rupees) — no currency conversion is performed.
- All users must be registered and authenticated to perform any transactional operation (cart, orders, reviews).
- Phone numbers follow Pakistani mobile format: 03XX-XXXXXXX.

13. Deliverables Checklist

Deliverable	DB Course	Status
11-table MySQL database (omnicart_db)	Required	Complete
3 stored procedures (PlaceOrder, UpdateProductInventory, DeactivateUser)	Required	Complete
2 additional procedures (ProcessPayment, CancelOrder)	Bonus	Complete
5 complex SQL queries with JOINS, aggregates, subqueries	Required	Complete
3 database views (CustomerOrderHistory, TopRatedProducts, VendorSalesSummary)	Required	Complete
Atomic transaction handling with ROLLBACK on error	Required	Complete
Foreign key constraints with ON DELETE / ON UPDATE rules	Required	Complete
Seed data: 5 categories, 3 vendors, 20+ products, 10 customers	Required	Complete
SVD collaborative filtering model trained and saved as .pkl	AI	Complete
TF-IDF content-based engine with cosine similarity	AI	Complete
Sentiment analysis pipeline — scores in reviews table	AI	Complete
GPT-4 recommendation explanations	AI	Complete
4 n8n automation workflows active	AI	Complete
React frontend — 3 portals (Customer, Seller, Admin)	Full Stack	Complete
ERD generated from MySQL Workbench	Required	Pending

References

- OmniCart Project Proposal — CS 2005: Database Systems, Spring 2026.
- OmniCart 35-Day Implementation Plan — CS 2005 / AI, Spring 2026.
- MySQL 8.0 Reference Manual — dev.mysql.com/doc

- Surprise Library Documentation — surpriselib.com
- Scikit-learn TfidfVectorizer — scikit-learn.org
- TextBlob Documentation — textblob.readthedocs.io
- n8n Documentation — docs.n8n.io
- Amazon Product Reviews Dataset — Kaggle (used for model training only)